

Feature Extraction

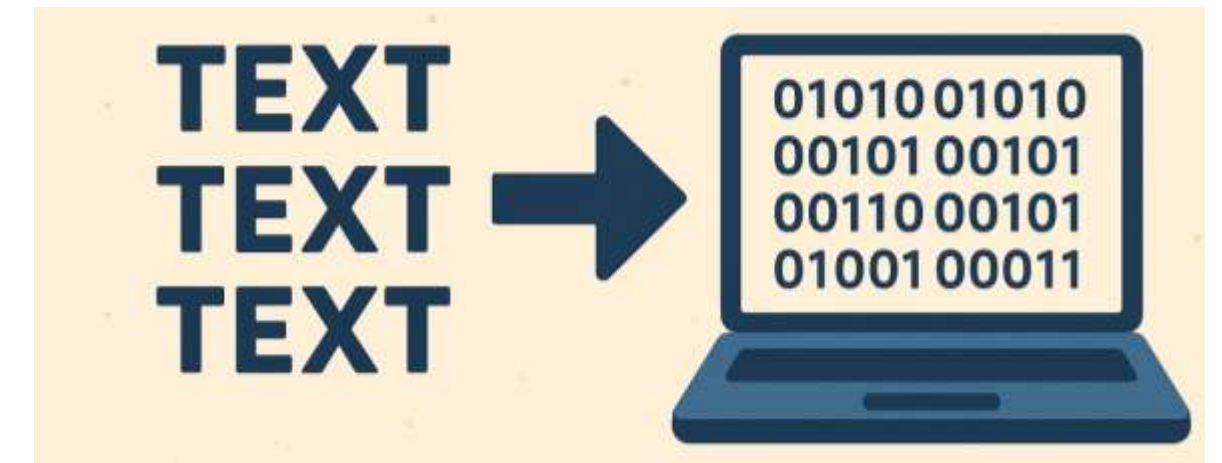
Muhammad Yazid Supriadi, S.Kom, M.Kom
STT Terpadu Nurul Fikri

Tujuan Perkuliahan

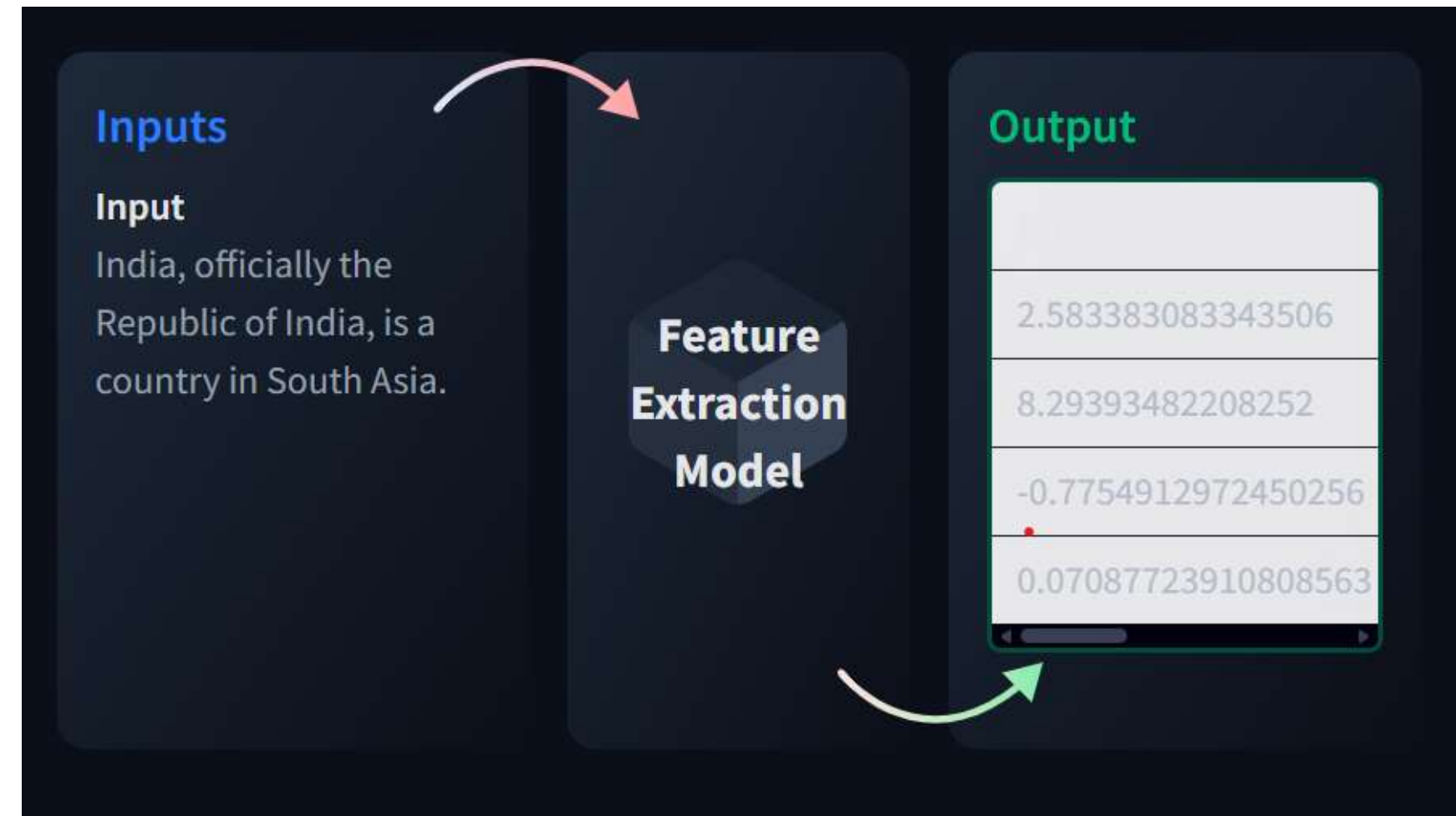
- Memahami konsep feature extraction NLP
- Menjelaskan metode representasi teks
- Mengimplementasikan teknik feature extraction
- Mengetahui dan mengimplementasikan beberapa metode feature extraction

Komputer tidak paham teks

- Komputer tidak memahami bahasa manusia
- Pada dasarnya, komputer hanya memahami angka.
- Teks adalah data yang tidak terstruktur, ambigu dan kompleks.



Feature Extraction



- Feature extraction merupakan proses mengubah data mentah (teks) menjadi representasi numerik(matrix atau vektor).
- Menurut Christopher D. Manning & Hinrich Schütze dalam buku *Foundations of Statistical Natural Language Processing*:
representasi teks adalah langkah fundamental dalam pemodelan bahasa karena menentukan bagaimana informasi linguistik ditangkap oleh model.

Feature Extraction Methods

1. Frequency-Based Methods : Bag of Words(BoW), TF-IDF
2. Sequence-Based Methods : N-grams
3. Character-Level Methods : character n-grams
4. Embedding Based Methods : Word2Vec, GloVe, FastText
5. Contextual Embedding: BERT

Bag Of Words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

Bag of Words (BoW) merupakan salah satu metode ekstraksi fitur yang merepresentasikan teks berdasarkan frekuensi kemunculan kata, tanpa memperhatikan urutan kata.

Dalam metode BoW, setiap kata pada dokumen dianggap sebagai fitur dan dihitung frekuensi kemunculannya.

Sebagai contoh ada 3 kalimat review yang akan diproses menggunakan BoW:

- 1. kopi ini enak banget
- 2. kopi ini pahit banget
- 3. kopi ini tidak enak

Review	kopi	ini	enak	banget	pahit	tidak
kopi ini enak banget	1	1	1	1	0	0
kopi ini pahit banget	1	1	0	1	1	0
kopi ini tidak enak	1	1	1	0	0	1

Keuntungan menggunakan Bag Of Words

- Mudah diimplementasikan dan tidak membutuhkan komputasi yang kompleks
- Bag of Word dapat diimplementasikan di berbagai macam tugas dalam NLP seperti text classification, sentiment analysis, document clustering, dll.
- Metode BoW dapat bekerja dengan baik dengan data terbatas.
- Vektor yang dihasilkan dari proses BoW dapat ditafsirkan sehingga memudahkan untuk memahami kata-kata mana yang paling penting dalam sebuah dokumen.

Kelemahan menggunakan Bag Of Words

- Tidak memahami konteks
- Mengabaikan urutan kata
- Dimensi sangat besar (High Dimensionality)
- Tidak menangkap makna
- Sparse matrix : jika punya ribuan bahkan jutaan kata, maka setiap dokumen akan direpresentasikan sebagai vector dengan Panjang ribuan atau jutaan yang Sebagian besar elemennya bernilai 0.
- Kata baru yang muncul tidak dikenali, sehingga variasi kata dianggap berbeda.

TF-IDF

Term Frequency - Inverse Document Frequency

- TF-IDF merupakan gabungan dari 2 komponen yaitu:
- TF (Term Frequency) : mengukur frekuensi kemunculan kata dalam suatu dokumen.
- Inverse Document Frequency (IDF): mengukur tingkat kelangkaan kata dalam keseluruhan corpus.
- Kombinasi keduanya digunakan untuk menentukan tingkat kepentingan suatu kata dalam dokumen.
- TF-IDF dikembangkan dari BoW untuk mengurangi pengaruh kata umum dan menonjolkan kata yang benar-benar penting dalam dokumen.

Term Frequency (TF)

Mengukur frekuensi kemunculan term dalam dokumen:

$$TF(t, d) = \frac{f(t, d)}{\sum_{t'} f(t', d)}$$

Keterangan:

$f(t, d)$: jumlah kemunculan term t di dokumen d

Penyebut: total semua kata dalam dokumen

Inverse Document Frequency (IDF)

Mengukur kelangkaan term dalam corpus:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

Keterangan:

N : jumlah dokumen

$df(t)$:jumlah dokumen yang mengandung term t

NB: Semakin kecil $df(t)$ → semakin besar IDF

TF-IDF

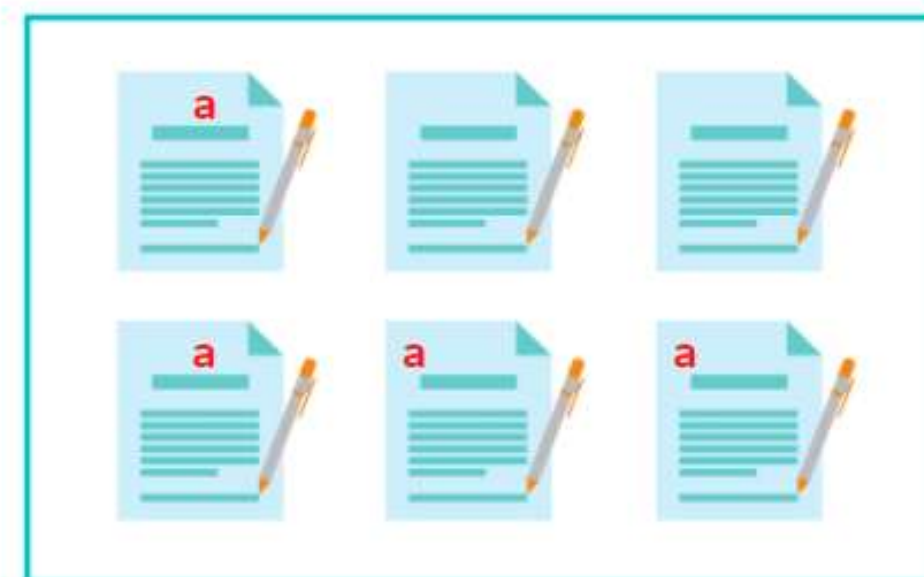
$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

TF



Frequency of a word
within the document

IDF



Frequency of a word
across the documents

Contoh terdapat 3 review sebagai berikut:

- **R1:** kopi ini enak
 - **R2:** kopi ini tidak enak
 - **R3:** kopi mahal
-
- Vocabulary yang terbentuk:
[kopi, ini, enak, tidak, mahal]

Hitung TF (Normalized)

Kata	R1	R2	R3	TF1	TF2	TF3
kopi	1	1	1	$1/3 = 0.33$	$1/4 = 0.25$	$1/2 = 0.5$
ini	1	1	0	$1/3 = 0.33$	$1/4 = 0.25$	0
enak	1	1	0	$1/3 = 0.33$	$1/4 = 0.25$	0
tidak	0	1	0	$0/3 = 0$	$1/4 = 0.25$	0
mahal	0	0	1	$0/3 = 0$	0	$1/2 = 0.5$

Hitung IDF

Karena terdapat 3 dokumen review maka jumlah dokumen = 3
 $IDF = \log(3/df)$

Kata	R1	R2	R3	TF1	TF2	TF3	IDF
kopi	1	1	1	$1/3 = 0.33$	$1/4 = 0.25$	$1/2 = 0.5$	$\log(3/3) = 0$
ini	1	1	0	$1/3 = 0.33$	$1/4 = 0.25$	0	$\log(3/2) = 0.176$
enak	1	1	0	$1/3 = 0.33$	$1/4 = 0.25$	0	$\log(3/2) = 0.176$
tidak	0	1	0	$0/3 = 0$	$1/4 = 0.25$	0	$\log(3/1) = 0.477$
mahal	0	0	1	$0/3 = 0$	0	$1/2 = 0.5$	$\log(3/1) = 0.477$

Nilai TF-IDF:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Kata	R1	R2	R3	TF1	TF2	TF3	IDF	TF-IDF1	TF-IDF2	TF-IDF3
kopi	1	1	1	$1/3 = 0.33$	$1/4 = 0.25$	$1/2 = 0.5$	$\log(3/3) = 0$	0	0	0
ini	1	1	0	$1/3 = 0.33$	$1/4 = 0.25$	0	$\log(3/2) = 0.176$	0.058	0.044	0
enak	1	1	0	$1/3 = 0.33$	$1/4 = 0.25$	0	$\log(3/2) = 0.176$	0.058	0.044	0
tidak	0	1	0	$0/3 = 0$	$1/4 = 0.25$	0	$\log(3/1) = 0.477$	0	0.119	0
mahal	0	0	1	$0/3 = 0$	0	$1/2 = 0.5$	$\log(3/1) = 0.477$	0	0	0.119

Dari 3 review yang telah diolah dengan TF-IDF menghasilkan nilai vector:

R1: kopi ini enak -> $R1 = [0, 0.058, 0.058, 0, 0]$
R2: kopi ini tidak enak -> $R2 = [0, 0.044, 0.044, 0.119, 0]$
R3: kopi mahal -> $R3 = [0, 0, 0, 0, 0.239]$

Dokumen	Kata Dominan	Makna
R1	ini, enak	positif
R2	tidak	negatif
R3	mahal	harga

Keunggulan menggunakan TF-IDF

- Menangkap kata yang penting
- Mengurangi pengaruh stop words
- Dapat membantu model dalam klasifikasi teks, pencarian dokumen, clustering
- Mudah digunakan dan diinterpretasi
- Efisien secara Komputasi dan scalable
- Cocok untuk Data Kecil–Menengah

Kekurangan menggunakan TF-IDF

- Tidak Memahami Konteks
- Tidak cocok untuk dokumen pendek
- Mengabaikan urutan kata
- Tidak Menangkap Makna (Semantik)
- Dimensi Tinggi (High Dimensionality)
- Sparse Matrix (Banyak Nol)
- Tidak Menangani Out-of-Vocabulary (OOV)
- Tidak Menangkap Konteks Global yang Kompleks
- Sensitif terhadap Preprocessing

N-gram

- N-gram Adalah representasi teks berupa **urutan kontigu (contiguous sequence) dari N unit linguistik** (biasanya kata atau karakter) dalam suatu korpus.

Menurut Speech and Language Processing:

N-gram merupakan model probabilistik sederhana yang digunakan untuk merepresentasikan dan memodelkan urutan bahasa alami berdasarkan distribusi lokal.

Konsep dasar n-gram:

Jika diberikan sebuah urutan kata:

$w_1, w_2, w_3, \dots, w_n$

Maka:

Unigram (N=1) $\rightarrow w_1, w_2, w_3$

Bigram (N=2) $\rightarrow (w_1, w_2), (w_2, w_3)$

Trigram (N=3) $\rightarrow (w_1, w_2, w_3)$

Dalam *language modeling*, N-gram digunakan untuk memperkirakan probabilitas suatu kalimat:

$$P(w_1, w_2, \dots, w_n)$$

Karena kompleks, digunakan pendekatan **Markov assumption**:

Bigram Model

$$P(w_n \mid w_{n-1})$$

Trigram Model

$$P(w_n \mid w_{n-2}w_{n-1})$$

Contoh jenis-jenis n-gram

N	Nama	Contoh dari “Saya tidak suka kopi”
1	Unigram	saya, tidak, suka, kopi
2	Bigram	(saya tidak), (tidak suka), (suka kopi)
3	Trigram	(saya tidak suka), (tidak suka kopi)
4+	4-gram dst.	Semakin besar n → semakin spesifik, semakin jarang

Contoh N-gram

Sebagai contoh kita memiliki corpus sederhana berisikan :

Kalimat 1: “saya tidak suka kopi”

Kalimat 2: “saya suka teh”

Kalimat 3: “dia tidak suka kopi”

Lalu kita hitung probabilitas kalimat target “saya tidak suka kopi” menggunakan Unigram, Bigram, dan Trigram dengan metode Maximum Likelihood Estimation (MLE).

Frekuensi Unigram di seluruh corpus:

Kata	Frekuensi (Count)	Probabilitas P(w)
saya	2	$2/11 \approx \mathbf{0.1818}$
tidak	2	$2/11 \approx \mathbf{0.1818}$
suka	3	$3/11 \approx \mathbf{0.2727}$
kopi	2	$2/11 \approx \mathbf{0.1818}$
teh	1	$1/11 \approx \mathbf{0.0909}$
dia	1	$1/11 \approx \mathbf{0.0909}$

Probabilitas keseluruhan kalimat “saya tidak suka kopi”:

$$P = P(saya) \times P(tidak) \times P(suka) \times P(kopi)$$

$$= \frac{2}{11} \times \frac{2}{11} \times \frac{3}{11} \times \frac{2}{11} = \frac{24}{14641} \approx \mathbf{0.001639}$$

Bigram yang muncul di corpus:

- (saya, tidak) $\rightarrow 1$
- (tidak, suka) $\rightarrow 2$
- (suka, kopi) $\rightarrow 2$
- (saya, suka) $\rightarrow 1$
- (suka, teh) $\rightarrow 1$
- (dia, tidak) $\rightarrow 1$

Probabilitas Kondisional Bigram yang relevan
untuk kalimat kita:

- $P(\text{tidak} \mid \text{saya}) = 1/2 = \mathbf{0.5000}$
- $P(\text{suka} \mid \text{tidak}) = 2/2 = \mathbf{1.0000}$
- $P(\text{kopi} \mid \text{suka}) = 2/3 \approx \mathbf{0.6667}$

Probabilitas keseluruhan kalimat dengan Bigram:

$$\begin{aligned} &P(\textit{saya tidak suka kopi}) \\ &= P(\textit{saya}) \times P(\textit{tidak}|\textit{saya}) \times P(\textit{suka}|\textit{tidak}) \times P(\textit{kopi}|\textit{suka}) \\ &= \frac{2}{11} \times \frac{1}{2} \times 1 \times \frac{2}{3} = \frac{2}{33} \approx \mathbf{0.0606} \end{aligned}$$

Trigram yang muncul di corpus:

(saya, tidak, suka) $\rightarrow$ 1

(tidak, suka, kopi) $\rightarrow$ 2

(saya, suka, teh) $\rightarrow$ 1

(dia, tidak, suka) $\rightarrow$ 1

Probabilitas Kondisional Trigram yang relevan:

$P(\text{suka} \mid \text{saya tidak}) = 1/1 = \mathbf{1.0000}$

$P(\text{kopi} \mid \text{tidak suka}) = 2/2 = \mathbf{1.0000}$

Probabilitas keseluruhan kalimat dengan Trigram:

$P(\text{saya tidak suka kopi})$

$= P(\text{saya}) \times P(\text{tidak}|\text{saya}) \times P(\text{suka}|\text{saya tidak}) \times P(\text{kopi}|\text{tidak suka})$

$$= \frac{2}{11} \times \frac{1}{2} \times 1 \times 1 = \frac{1}{11} \approx \mathbf{0.0909}$$

- Autocomplete seperti pada keyboard HP
- Spell checking
- Speech recognition
- Machine Translation
- Plagiarism detection
- Sentiment analysis



A web form consisting of a label 'Country' and an adjacent empty text input field. A mouse cursor is positioned over the input field.

N-gram sering menjadi pilihan terbaik ketika:

- Resource terbatas,
- Butuh kecepatan tinggi,
- Perlu interpretasi yang jelas,
- Data tidak terlalu banyak.
- Menangkap konteks local
- Bahasa dengan corpus kecil

Kekurangan n-gram

- Data Sparsity
Banyak n-gram tidak muncul di corpus, sehingga banyak menampilkan probabilitas 0
- Curse of Dimensionality
Jumlah kemungkinan n-gram tumbuh eksponensial sehingga tidak scalable, butuh data sangat besar
- Fixed Context Window
Hanya melihat n-1 kata sebelumnya, sehingga gagal tangkap long-range dependency
- Lack of Semantics & Generalization
n-gram hanya statistic permukaan, bukan memahami makna secara kompleks
- High Resource Consumption
untuk n besar membutuhkan resource dan komputasi yang besar

Thank you guys!